

Scheduling o planificación de procesos

Sistemas Operativos
DC - FCEN - UBA

27 de agosto de 2026

300

días sin que se cumpla

la Ley de
Financiamiento
Universitario

CiN

Consejo
Interuniversitario
Nacional



Universidad
Pública
Argentina

En Exactas, desde **diciembre 2023**
cada **dos días** se va un docente.

Convocatoria abierta

NMUN UBA

NATIONAL MODEL UNITED NATIONS
DELEGACIÓN UNIVERSIDAD DE BUENOS AIRES



PROLAG
PROCESO DE REGULACIÓN DE LA ACTIVIDAD DE LOS AGENTES
UNIVERSIDAD DE BUENOS AIRES



UBA
Universidad de
Buenos Aires

.UBAINTERNACIONAL

Convocatoria abierta

NMUN UBA

National Model United Nations

Delegación Universidad de Buenos Aires

internacionales.uba.ar

Menú para hoy

- 1 Repaso
- 2 Conceptos básicos
- 3 Criterios y objetivos
- 4 Algoritmos de scheduling
- 5 Ejercicio de diseño
- 6 Scheduling de Tiempo Real
- 7 El simulador del TP
- 8 Scheduling en la vida real
- 9 Cierre

Menú para hoy

- 1 Repaso
- 2 Conceptos básicos
- 3 Criterios y objetivos
- 4 Algoritmos de scheduling
- 5 Ejercicio de diseño
- 6 Scheduling de Tiempo Real
- 7 El simulador del TP
- 8 Scheduling en la vida real
- 9 Cierre

- Multiprogramación:

- Multiprogramación: Tener procesos corriendo todo el tiempo para maximizar la utilización de CPU.

- Multiprogramación: Tener procesos corriendo todo el tiempo para maximizar la utilización de CPU.
- Cuando los procesos “llegan”, se inicializa su **PCB**.

- Multiprogramación: Tener procesos corriendo todo el tiempo para maximizar la utilización de CPU.
- Cuando los procesos “llegan”, se inicializa su **PCB**.
- Sólo 1 proceso puede correr a la vez (1 core), los demás deben esperar.

- Multiprogramación: Tener procesos corriendo todo el tiempo para maximizar la utilización de CPU.
- Cuando los procesos “llegan”, se inicializa su **PCB**.
- Sólo 1 proceso puede correr a la vez (1 core), los demás deben esperar.
- Muchos procesos (o sea, sus PCBs) se mantienen en memoria al mismo tiempo. Cuando la CPU se libera, se debe elegir a otro proceso de la cola de *ready* y darle la CPU

- Multiprogramación: Tener procesos corriendo todo el tiempo para maximizar la utilización de CPU.
- Cuando los procesos “llegan”, se inicializa su **PCB**.
- Sólo 1 proceso puede correr a la vez (1 core), los demás deben esperar.
- Muchos procesos (o sea, sus PCBs) se mantienen en memoria al mismo tiempo. Cuando la CPU se libera, se debe elegir a otro proceso de la cola de *ready* y darle la CPU → **CPU scheduling**.

Repaso

Estados de un proceso

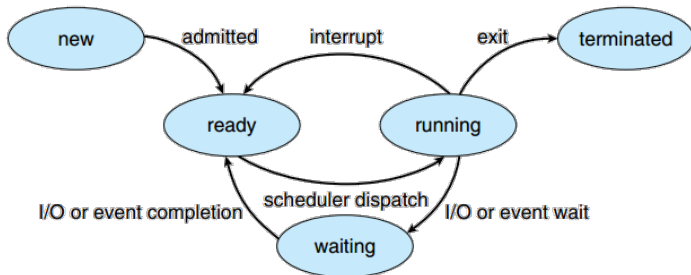


Figura: Diagrama de estados de un proceso¹

¹Operating Systems Concepts, Abraham Silberschatz & Peter B. Galvin. ▶

Menú para hoy

- 1 Repaso
- 2 Conceptos básicos**
- 3 Criterios y objetivos
- 4 Algoritmos de scheduling
- 5 Ejercicio de diseño
- 6 Scheduling de Tiempo Real
- 7 El simulador del TP
- 8 Scheduling en la vida real
- 9 Cierre

Ráfagas de I/O y CPU

- La ejecución de un proceso consiste en un **ciclo**, entre ejecución de CPU y espera por I/O.

Ráfagas de I/O y CPU

- La ejecución de un proceso consiste en un **ciclo**, entre ejecución de CPU y espera por I/O.
- La ejecución siempre comienza con una **ráfaga de CPU**. Luego le sigue una **ráfaga de I/O**. Luego CPU, luego I/O, luego CPU... y así hasta que termina.

Ráfagas de I/O y CPU

- La ejecución de un proceso consiste en un **ciclo**, entre ejecución de CPU y espera por I/O.
- La ejecución siempre comienza con una **ráfaga de CPU**. Luego le sigue una **ráfaga de I/O**. Luego CPU, luego I/O, luego CPU... y así hasta que termina.
- Un programa **intensivo en I/O** típicamente tiene muchas ráfagas de CPU cortitas.
- Un programa **intensivo en CPU** suele tener unas pocas ráfagas de CPU largas.

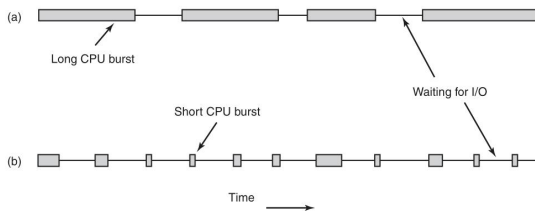
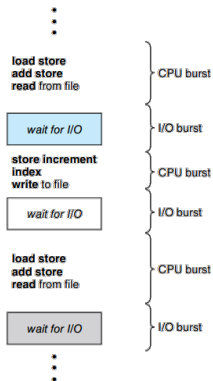


Figura: Secuencia de ráfagas de CPU y de I/O. (a) Un proceso intensivo en CPU. (b) Un proceso intensivo en I/O.²

Ráfagas de I/O y CPU

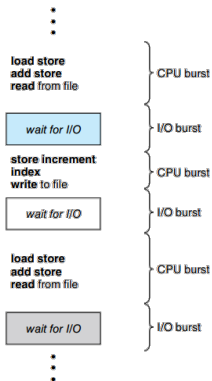
Identificar las ráfagas “a mano”



- Dada una traza de lo que hace un proceso, queremos separar **ráfagas de CPU** de **ráfagas de I/O**.

Ráfagas de I/O y CPU

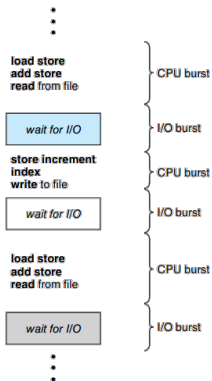
Identificar las ráfagas “a mano”



- Dada una traza de lo que hace un proceso, queremos separar **ráfagas de CPU** de **ráfagas de I/O**.
- La regla es simple: si el proceso está **usando el procesador** es ráfaga de CPU; si está **esperando a un dispositivo**, es ráfaga de I/O.

Ráfagas de I/O y CPU

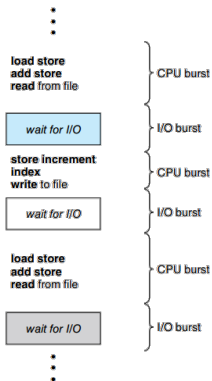
Identificar las ráfagas “a mano”



- Dada una traza de lo que hace un proceso, queremos separar **ráfagas de CPU** de **ráfagas de I/O**.
- La regla es simple: si el proceso está **usando el procesador** es ráfaga de CPU; si está **esperando a un dispositivo**, es ráfaga de I/O.

Ráfagas de I/O y CPU

Identificar las ráfagas “a mano”



- Dada una traza de lo que hace un proceso, queremos separar **ráfagas de CPU** de **ráfagas de I/O**.
- La regla es simple: si el proceso está **usando el procesador** es ráfaga de CPU; si está **esperando a un dispositivo**, es ráfaga de I/O.
- Acá se alternan ráfagas cortas de CPU con esperas largas → proceso **intensivo en I/O**.

Éste es exactamente el **ejercicio 1** de la guía 2.

CPU Scheduler o Planificador de CPU

- El **scheduler del Sistema Operativo** es responsable de seleccionar un proceso de todos los que están en memoria listos para ejecutar, y darle la CPU para que procese.

CPU Scheduler o Planificador de CPU

- El **scheduler del Sistema Operativo** es responsable de seleccionar un proceso de todos los que están en memoria listos para ejecutar, y darle la CPU para que procese.
- *Preemptive* o *nonpreemptive*, esa es la cuestión.

CPU Scheduler o Planificador de CPU

- El **scheduler del Sistema Operativo** es responsable de seleccionar un proceso de todos los que están en memoria listos para ejecutar, y darle la CPU para que procese.
- *Preemptive* o *nonpreemptive*, esa es la cuestión.
 - **Nonpreemptive** o **cooperativo**:

CPU Scheduler o Planificador de CPU

- El **scheduler del Sistema Operativo** es responsable de seleccionar un proceso de todos los que están en memoria listos para ejecutar, y darle la CPU para que procese.
- *Preemptive* o *nonpreemptive*, esa es la cuestión.
 - **Nonpreemptive** o **cooperativo**: Sin desalojo. Una vez que un proceso obtiene la CPU, ejecuta hasta liberarla de forma voluntaria

CPU Scheduler o Planificador de CPU

- El **scheduler del Sistema Operativo** es responsable de seleccionar un proceso de todos los que están en memoria listos para ejecutar, y darle la CPU para que procese.
- *Preemptive* o *nonpreemptive*, esa es la cuestión.
 - **Nonpreemptive** o **cooperativo**: Sin desalojo. Una vez que un proceso obtiene la CPU, ejecuta hasta liberarla de forma voluntaria → si termina o si pasa a estado *waiting*.

CPU Scheduler o Planificador de CPU

- El **scheduler del Sistema Operativo** es responsable de seleccionar un proceso de todos los que están en memoria listos para ejecutar, y darle la CPU para que procese.
- *Preemptive* o *nonpreemptive*, esa es la cuestión.
 - **Nonpreemptive** o **cooperativo**: Sin desalojo. Una vez que un proceso obtiene la CPU, ejecuta hasta liberarla de forma voluntaria → si termina o si pasa a estado *waiting*.
 - **Preemptive** o **no cooperativo**:

- El **scheduler del Sistema Operativo** es responsable de seleccionar un proceso de todos los que están en memoria listos para ejecutar, y darle la CPU para que procese.
- *Preemptive* o *nonpreemptive*, esa es la cuestión.
 - **Nonpreemptive** o **cooperativo**: Sin desalojo. Una vez que un proceso obtiene la CPU, ejecuta hasta liberarla de forma voluntaria → si termina o si pasa a estado *waiting*.
 - **Preemptive** o **no cooperativo**: Con desalojo. El scheduler puede determinar cuándo sacarle la CPU a un proceso.

Momento para preguntas sobre los conceptos enunciados

Menú para hoy

- 1 Repaso
- 2 Conceptos básicos
- 3 Criterios y objetivos**
- 4 Algoritmos de scheduling
- 5 Ejercicio de diseño
- 6 Scheduling de Tiempo Real
- 7 El simulador del TP
- 8 Scheduling en la vida real
- 9 Cierre

Criterios y objetivos

Criterio	Descripción	Objetivo

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU		

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput		

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	MAXIMIZAR ↑

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	MAXIMIZAR ↑
Turnaround		

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	MAXIMIZAR ↑
Turnaround	Cuánto le toma a un proceso terminar de ejecutar	

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	MAXIMIZAR ↑
Turnaround	Cuánto le toma a un proceso terminar de ejecutar (tiempo esperando en <i>ready</i> + tiempo ejecutando en CPU + tiempo haciendo I/O).	

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	MAXIMIZAR ↑
Turnaround	Cuánto le toma a un proceso terminar de ejecutar (tiempo esperando en <i>ready</i> + tiempo ejecutando en CPU + tiempo haciendo I/O).	MINIMIZAR ↓

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	MAXIMIZAR ↑
Turnaround	Cuánto le toma a un proceso terminar de ejecutar (tiempo esperando en <i>ready</i> + tiempo ejecutando en CPU + tiempo haciendo I/O).	MINIMIZAR ↓
Waiting time		

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	MAXIMIZAR ↑
Turnaround	Cuánto le toma a un proceso terminar de ejecutar (tiempo esperando en <i>ready</i> + tiempo ejecutando en CPU + tiempo haciendo I/O).	MINIMIZAR ↓
Waiting time	Suma de los periodos en <i>ready</i> .	

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	MAXIMIZAR ↑
Turnaround	Cuánto le toma a un proceso terminar de ejecutar (tiempo esperando en <i>ready</i> + tiempo ejecutando en CPU + tiempo haciendo I/O).	MINIMIZAR ↓
Waiting time	Suma de los periodos en <i>ready</i> .	MINIMIZAR ↓

Crterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	MAXIMIZAR ↑
Turnaround	Cuánto le toma a un proceso terminar de ejecutar (tiempo esperando en <i>ready</i> + tiempo ejecutando en CPU + tiempo haciendo I/O).	MINIMIZAR ↓
Waiting time	Suma de los periodos en <i>ready</i> .	MINIMIZAR ↓
Response time		

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	MAXIMIZAR ↑
Turnaround	Cuánto le toma a un proceso terminar de ejecutar (tiempo esperando en <i>ready</i> + tiempo ejecutando en CPU + tiempo haciendo I/O).	MINIMIZAR ↓
Waiting time	Suma de los periodos en <i>ready</i> .	MINIMIZAR ↓
Response time	Tiempo transcurrido desde que el proceso es “lanzado” (desde que se encuentra en <i>idle</i> por primera vez) HASTA la primera vez que pasa a estar en <i>running</i>	

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	MAXIMIZAR ↑
Turnaround	Cuánto le toma a un proceso terminar de ejecutar (tiempo esperando en <i>ready</i> + tiempo ejecutando en CPU + tiempo haciendo I/O).	MINIMIZAR ↓
Waiting time	Suma de los periodos en <i>ready</i> .	MINIMIZAR ↓
Response time	Tiempo transcurrido desde que el proceso es “lanzado” (desde que se encuentra en <i>idle</i> por primera vez) HASTA la primera vez que pasa a estar en <i>running</i> ¡No es lo mismo que turnaround!	

Criterios y objetivos

Criterio	Descripción	Objetivo
Uso de CPU	Mantener la CPU tan ocupada como sea posible.	MAXIMIZAR ↑
Throughput	Cantidad de procesos terminados por unidad de tiempo.	MAXIMIZAR ↑
Turnaround	Cuánto le toma a un proceso terminar de ejecutar (tiempo esperando en <i>ready</i> + tiempo ejecutando en CPU + tiempo haciendo I/O).	MINIMIZAR ↓
Waiting time	Suma de los periodos en <i>ready</i> .	MINIMIZAR ↓
Response time	Tiempo transcurrido desde que el proceso es “lanzado” (desde que se encuentra en <i>idle</i> por primera vez) HASTA la primera vez que pasa a estar en <i>running</i> ¡No es lo mismo que turnaround!	MINIMIZAR ↓

¿Cómo sabemos que una política de scheduling es mejor que otra?

¿Cómo sabemos que una política de scheduling es mejor que otra?

- Hay que definir los criterios para comparar.

¿Cómo sabemos que una política de scheduling es mejor que otra?

- Hay que definir los criterios para comparar.
- Para medir bien, deberíamos considerar muchos procesos con muchas ráfagas de CPU y muchas ráfagas de I/O.

¿Cómo sabemos que una política de scheduling es mejor que otra?

- Hay que definir los criterios para comparar.
- Para medir bien, deberíamos considerar muchos procesos con muchas ráfagas de CPU y muchas ráfagas de I/O.
- Por simplicidad, en la materia vamos a considerar sólo algunas **ráfagas de CPU** por proceso, y comparar por **waiting time promedio** y el **turnaround promedio** (salvo que se indique otra cosa).

¿Cómo sabemos que una política de scheduling es mejor que otra?

- Hay que definir los criterios para comparar.
- Para medir bien, deberíamos considerar muchos procesos con muchas ráfagas de CPU y muchas ráfagas de I/O.
- Por simplicidad, en la materia vamos a considerar sólo algunas **ráfagas de CPU** por proceso, y comparar por **waiting time promedio** y el **turnaround promedio** (salvo que se indique otra cosa).
- Si bien la mayoría de las arquitecturas de CPU actuales son *multicore*, trabajaremos en el contexto de un **único core disponible**.

Menú para hoy

- 1 Repaso
- 2 Conceptos básicos
- 3 Criterios y objetivos
- 4 Algoritmos de scheduling**
- 5 Ejercicio de diseño
- 6 Scheduling de Tiempo Real
- 7 El simulador del TP
- 8 Scheduling en la vida real
- 9 Cierre

First-Come, First-Served (FCFS)

Definición

- Se da el procesador al **primer proceso** que lo pide.

First-Come, First-Served (FCFS)

Definición

- Se da el procesador al **primer proceso** que lo pide.
- Es *nonpreemptive*.

First-Come, First-Served (FCFS)

Ejemplo

Considerar los siguientes procesos, llegados en el instante 0 y ordenados por el scheduler tal como se muestra en la tabla y con el largo de la ráfaga de CPU indicada en milisegundos:

Proceso	Ráfaga de CPU (ms)
P1	24
P2	3
P3	3

First-Come, First-Served (FCFS)

Ejemplo

Considerar los siguientes procesos, llegados en el instante 0 y ordenados por el scheduler tal como se muestra en la tabla y con el largo de la ráfaga de CPU indicada en milisegundos:

Proceso	Ráfaga de CPU (ms)
P1	24
P2	3
P3	3

Diagrama de Gantt:



- Waiting time promedio: $(0 + 24 + 27)/3 = 17$ milisegundos
- Turnaround promedio: $(24 + 27 + 30)/3 = 27$ milisegundos

¿Cómo se construye un diagrama de Gantt?

El diagrama de Gantt es **la** herramienta de toda la guía. Se construye así:

- 1 Un **eje horizontal de tiempo**, con una marca por unidad.

¿Cómo se construye un diagrama de Gantt?

El diagrama de Gantt es **la** herramienta de toda la guía. Se construye así:

- 1 Un **eje horizontal de tiempo**, con una marca por unidad.
- 2 Un **bloque** por cada tramo continuo en que un proceso tiene la CPU. Un proceso desalojado y reanudado aparece en **varios bloques**.

¿Cómo se construye un diagrama de Gantt?

El diagrama de Gantt es **la** herramienta de toda la guía. Se construye así:

- 1 Un **eje horizontal de tiempo**, con una marca por unidad.
- 2 Un **bloque** por cada tramo continuo en que un proceso tiene la CPU. Un proceso desalojado y reanudado aparece en **varios bloques**.
- 3 Si en un instante **no hay ningún proceso listo**, la CPU queda **ociosa**: hay que dibujar el hueco.

¿Cómo se construye un diagrama de Gantt?

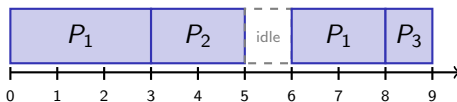
El diagrama de Gantt es **la** herramienta de toda la guía. Se construye así:

- 1 Un **eje horizontal de tiempo**, con una marca por unidad.
- 2 Un **bloque** por cada tramo continuo en que un proceso tiene la CPU. Un proceso desalojado y reanudado aparece en **varios bloques**.
- 3 Si en un instante **no hay ningún proceso listo**, la CPU queda **ociosa**: hay que dibujar el hueco.

¿Cómo se construye un diagrama de Gantt?

El diagrama de Gantt es la herramienta de toda la guía. Se construye así:

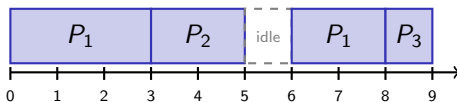
- 1 Un **eje horizontal de tiempo**, con una marca por unidad.
- 2 Un **bloque** por cada tramo continuo en que un proceso tiene la CPU. Un proceso desalojado y reanudado aparece en **varios bloques**.
- 3 Si en un instante **no hay ningún proceso listo**, la CPU queda **ociosa**: hay que dibujar el hueco.



¿Cómo se construye un diagrama de Gantt?

El diagrama de Gantt es la herramienta de toda la guía. Se construye así:

- 1 Un **eje horizontal de tiempo**, con una marca por unidad.
- 2 Un **bloque** por cada tramo continuo en que un proceso tiene la CPU. Un proceso desalojado y reanudado aparece en **varios bloques**.
- 3 Si en un instante **no hay ningún proceso listo**, la CPU queda **ociosa**: hay que dibujar el hueco.



Cómo se leen las métricas del diagrama

- **Turnaround** = instante en que **termina** – instante en que **llegó**.
- **Waiting time** = turnaround – ráfaga de CPU. **No** es el instante en que arranca, salvo que todos lleguen en $t = 0$.
- **Response time** = instante en que corre **por primera vez** – llegada.

First-Come, First-Served (FCFS)

Para pensar:

- ¿Cómo cambiarían las métricas si el orden de procesos fuese $P_2 - P_3 - P_1$?

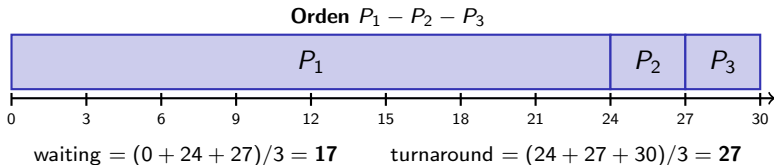
Proceso	Ráfaga de CPU (ms)
P_1	24
P_2	3
P_3	3

Los tres siguen llegando en $t = 0$ y nadie hace I/O. ¿El waiting time promedio sube, baja o queda igual? ¿Y el instante en que termina el último proceso?

First-Come, First-Served (FCFS)

¿Y si el orden fuese $P_2 - P_3 - P_1$?

Misma instancia ($P_1 = 24$, $P_2 = 3$, $P_3 = 3$, todos llegan en $t = 0$). Lo **único** que cambia es el orden en que entraron a la cola de *ready*.

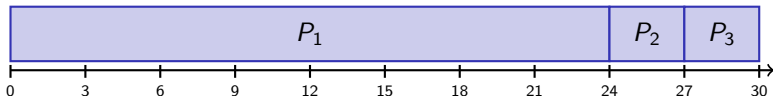


First-Come, First-Served (FCFS)

¿Y si el orden fuese $P_2 - P_3 - P_1$?

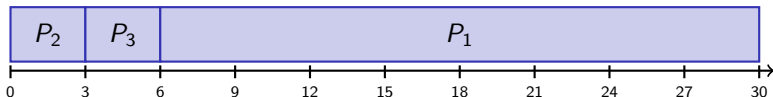
Misma instancia ($P_1 = 24$, $P_2 = 3$, $P_3 = 3$, todos llegan en $t = 0$). Lo **único** que cambia es el orden en que entraron a la cola de *ready*.

Orden $P_1 - P_2 - P_3$



$$\text{waiting} = (0 + 24 + 27)/3 = 17 \quad \text{turnaround} = (24 + 27 + 30)/3 = 27$$

Orden $P_2 - P_3 - P_1$



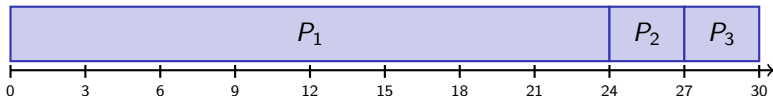
$$\text{waiting} = (0 + 3 + 6)/3 = 3 \quad \text{turnaround} = (3 + 6 + 30)/3 = 13$$

First-Come, First-Served (FCFS)

¿Y si el orden fuese $P_2 - P_3 - P_1$?

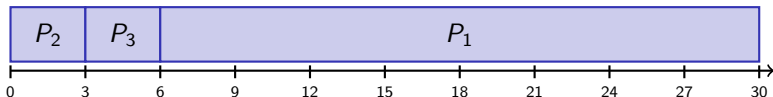
Misma instancia ($P_1 = 24$, $P_2 = 3$, $P_3 = 3$, todos llegan en $t = 0$). Lo **único** que cambia es el orden en que entraron a la cola de *ready*.

Orden $P_1 - P_2 - P_3$



$$\text{waiting} = (0 + 24 + 27)/3 = 17 \quad \text{turnaround} = (24 + 27 + 30)/3 = 27$$

Orden $P_2 - P_3 - P_1$



$$\text{waiting} = (0 + 3 + 6)/3 = 3 \quad \text{turnaround} = (3 + 6 + 30)/3 = 13$$

El trabajo total no cambió: la CPU nunca está ociosa y el último proceso termina en $t = 30$ en los dos casos. Lo que se desploma es el **promedio**, sólo por poner las ráfagas cortas primero: los 24 ms de P_1 dejan de sumarse a la espera de los demás. FCFS no elige ese orden — se lo impone la llegada. De ahí sale **SJF**.

First-Come, First-Served (FCFS)

Para pensar (cont.):

- ¿Qué pasaría si primero llegase un proceso intensivo en CPU y luego varios intensivos en I/O?

First-Come, First-Served (FCFS)

Para pensar (cont.):

- ¿Qué pasaría si primero llegase un proceso intensivo en CPU y luego varios intensivos en I/O? → *Convoy Effect*:

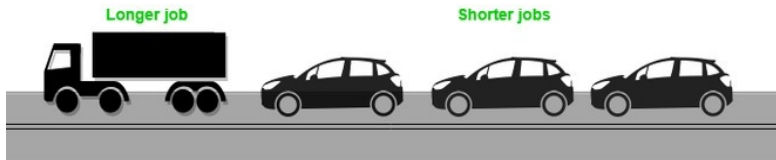


Figura: Convoy effect gráficamente

First-Come, First-Served (FCFS)

Para pensar (cont.):

- ¿Qué pasaría si primero llegase un proceso intensivo en CPU y luego varios intensivos en I/O? → *Convoy Effect*:

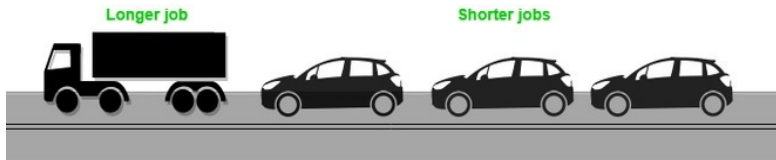


Figura: Convoy effect gráficamente

- ¿Qué pasaría si usásemos FCFS en un sistema interactivo?

Una instancia para comparar todo

Hasta acá todos los procesos llegaban en el instante 0. Eso casi nunca pasa. De ahora en adelante usamos **esta misma instancia** para todos los algoritmos, así podemos compararlos.

Proceso	Ráfaga de CPU	Instante de llegada
P_1	6	0
P_2	3	1
P_3	1	2
P_4	4	3

Una instancia para comparar todo

Hasta acá todos los procesos llegaban en el instante 0. Eso casi nunca pasa. De ahora en adelante usamos **esta misma instancia** para todos los algoritmos, así podemos compararlos.

Proceso	Ráfaga de CPU	Instante de llegada
P_1	6	0
P_2	3	1
P_3	1	2
P_4	4	3

Ojo con esto

Con llegadas distintas de 0, el **waiting time** ya no es el instante en que el proceso **arranca**. Hay que descontar lo que el proceso todavía no había llegado:

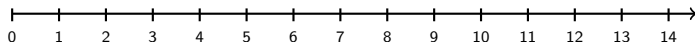
$$\text{waiting time} = \text{turnaround} - \text{ráfaga de CPU}$$

Es el error más común de la guía.

First-Come, First-Served (FCFS)

Ahora con instantes de llegada

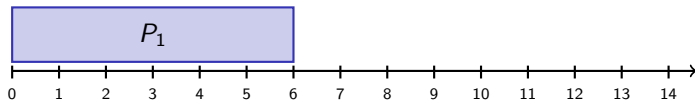
FCFS atiende en orden de llegada: P_1, P_2, P_3, P_4 .



First-Come, First-Served (FCFS)

Ahora con instantes de llegada

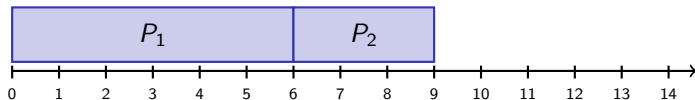
FCFS atiende en orden de llegada: P_1, P_2, P_3, P_4 .



First-Come, First-Served (FCFS)

Ahora con instantes de llegada

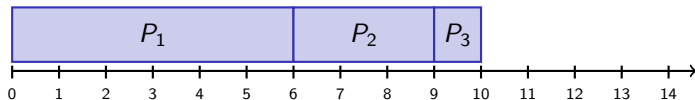
FCFS atiende en orden de llegada: P_1, P_2, P_3, P_4 .



First-Come, First-Served (FCFS)

Ahora con instantes de llegada

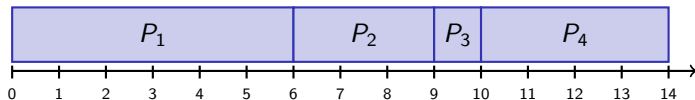
FCFS atiende en orden de llegada: P_1, P_2, P_3, P_4 .



First-Come, First-Served (FCFS)

Ahora con instantes de llegada

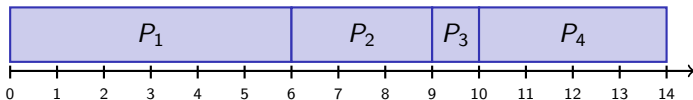
FCFS atiende en orden de llegada: P_1, P_2, P_3, P_4 .



First-Come, First-Served (FCFS)

Ahora con instantes de llegada

FCFS atiende en orden de llegada: P_1, P_2, P_3, P_4 .

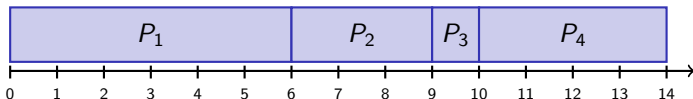


Proceso	Llegada	Fin	Turnaround	Waiting time
P_1	0	6	$6 - 0 = 6$	$6 - 6 = 0$
P_2	1	9	$9 - 1 = 8$	$8 - 3 = 5$
P_3	2	10	$10 - 2 = 8$	$8 - 1 = 7$
P_4	3	14	$14 - 3 = 11$	$11 - 4 = 7$

First-Come, First-Served (FCFS)

Ahora con instantes de llegada

FCFS atiende en orden de llegada: P_1, P_2, P_3, P_4 .



Proceso	Llegada	Fin	Turnaround	Waiting time
P_1	0	6	$6 - 0 = 6$	$6 - 6 = 0$
P_2	1	9	$9 - 1 = 8$	$8 - 3 = 5$
P_3	2	10	$10 - 2 = 8$	$8 - 1 = 7$
P_4	3	14	$14 - 3 = 11$	$11 - 4 = 7$

Turnaround promedio = $33/4 = 8,25$

Waiting time promedio = $19/4 = 4,75$

Round-Robin (RR)

Definición

- El scheduler va siguiendo la cola *ready* en orden de llegada de cada proceso, dándole la CPU a *cada proceso por un $quantum$* de tiempo.

Round-Robin (RR)

Definición

- El scheduler va siguiendo la cola *ready* en orden de llegada de cada proceso, dándole la CPU a *cada proceso por un quantum* de tiempo.
- Si el proceso que se está ejecutando consume su ráfaga de CPU antes de terminar su *quantum* (es decir, que ingresa en una ráfaga de I/O o el proceso termina por completo), deja la CPU voluntariamente y el scheduler le asigna la CPU al siguiente proceso.

Round-Robin (RR)

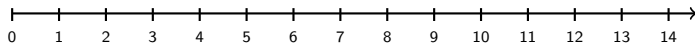
Definición

- El scheduler va siguiendo la cola *ready* en orden de llegada de cada proceso, dándole la CPU a *cada proceso por un quantum* de tiempo.
- Si el proceso que se está ejecutando consume su ráfaga de CPU antes de terminar su *quantum* (es decir, que ingresa en una ráfaga de I/O o el proceso termina por completo), deja la CPU voluntariamente y el scheduler le asigna la CPU al siguiente proceso.
- Si el proceso no termina su ráfaga de CPU al terminar el *quantum*, el scheduler lo desaloja y lo pone al final de la cola *ready*.

Round-Robin (RR)

Ejemplo con $quantum = 2$

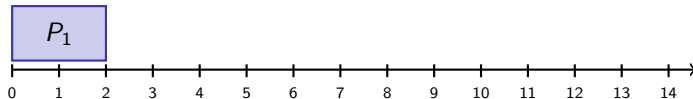
Misma instancia, $quantum = 2$. Lo importante es seguir la [cola ready](#).



Round-Robin (RR)

Ejemplo con $quantum = 2$

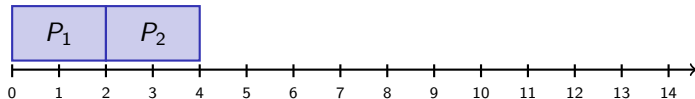
Misma instancia, $quantum = 2$. Lo importante es seguir la [cola ready](#).



Round-Robin (RR)

Ejemplo con $quantum = 2$

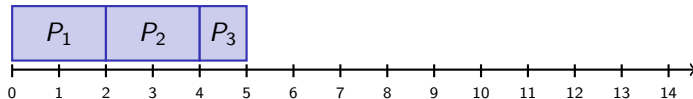
Misma instancia, $quantum = 2$. Lo importante es seguir la [cola ready](#).



Round-Robin (RR)

Ejemplo con $quantum = 2$

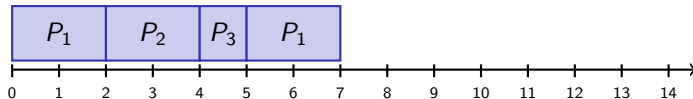
Misma instancia, $quantum = 2$. Lo importante es seguir la [cola ready](#).



Round-Robin (RR)

Ejemplo con $quantum = 2$

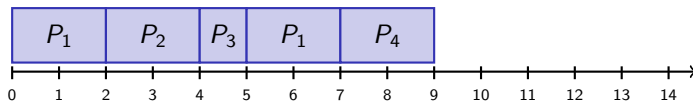
Misma instancia, $quantum = 2$. Lo importante es seguir la [cola ready](#).



Round-Robin (RR)

Ejemplo con $quantum = 2$

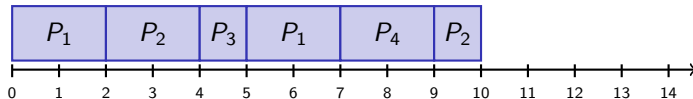
Misma instancia, $quantum = 2$. Lo importante es seguir la [cola ready](#).



Round-Robin (RR)

Ejemplo con $quantum = 2$

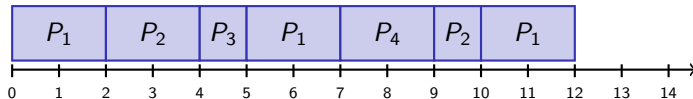
Misma instancia, $quantum = 2$. Lo importante es seguir la **cola ready**.



Round-Robin (RR)

Ejemplo con $quantum = 2$

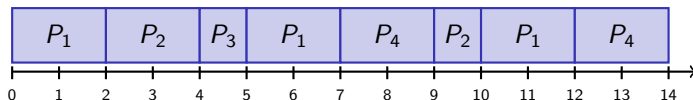
Misma instancia, $quantum = 2$. Lo importante es seguir la [cola ready](#).



Round-Robin (RR)

Ejemplo con $quantum = 2$

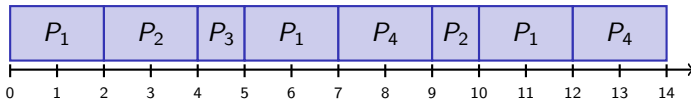
Misma instancia, $quantum = 2$. Lo importante es seguir la **cola ready**.



Round-Robin (RR)

Ejemplo con $quantum = 2$

Misma instancia, $quantum = 2$. Lo importante es seguir la **cola ready**.



Turnaround promedio = $35/4 = 8,75$
promedio = $7/4 = 1,75$

Waiting time promedio = $21/4 = 5,25$

Response

Round-Robin (RR)

Para pensar:

- ¿Qué pasa si el *quantum* dura mucho tiempo?

Round-Robin (RR)

Para pensar:

- ¿Qué pasa si el *quantum* dura mucho tiempo? → Se parece a FCFS.

Round-Robin (RR)

Para pensar:

- ¿Qué pasa si el *quantum* dura mucho tiempo? → Se parece a FCFS.
- ¿Qué pasa si el *quantum* dura muy poco?

Round-Robin (RR)

Para pensar:

- ¿Qué pasa si el *quantum* dura mucho tiempo? → Se parece a FCFS.
- ¿Qué pasa si el *quantum* dura muy poco? → Se va mucho tiempo en *context switch*.

Round-Robin (RR)

Para pensar:

- ¿Qué pasa si el *quantum* dura mucho tiempo? → Se parece a FCFS.
- ¿Qué pasa si el *quantum* dura muy poco? → Se va mucho tiempo en *context switch*.
- ¿Qué tan largo debe ser el *quantum* para tener buena *performance*?

Round-Robin (RR)

Para pensar:

- ¿Qué pasa si el *quantum* dura mucho tiempo? → Se parece a FCFS.
- ¿Qué pasa si el *quantum* dura muy poco? → Se va mucho tiempo en *context switch*.
- ¿Qué tan largo debe ser el *quantum* para tener buena *performance*?
→ Lo suficientemente grande como para sacar ventaja respecto al tiempo de *context switch*.

Shortest-Job-First (SJF)

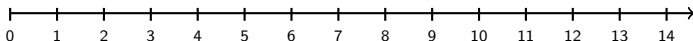
Definición

- Se asocia a cada proceso el **largo de su próxima ráfaga de CPU** y se elige para ejecutar al proceso que tenga la **menor**.

Shortest-Job-First (SJF)

Ejemplo (sin desalojo)

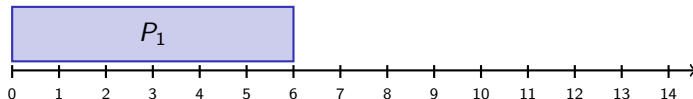
Misma instancia. En cada decisión elegimos, **entre los que ya llegaron**, el de ráfaga más corta. Sin desalojo: el que agarra la CPU la usa hasta terminar.



Shortest-Job-First (SJF)

Ejemplo (sin desalojo)

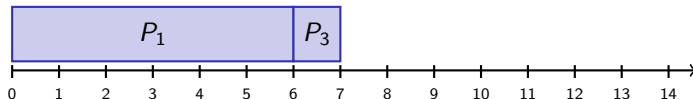
Misma instancia. En cada decisión elegimos, entre los que ya llegaron, el de ráfaga más corta. Sin desalojo: el que agarra la CPU la usa hasta terminar.



Shortest-Job-First (SJF)

Ejemplo (sin desalojo)

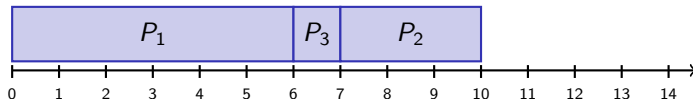
Misma instancia. En cada decisión elegimos, **entre los que ya llegaron**, el de ráfaga más corta. Sin desalojo: el que agarra la CPU la usa hasta terminar.



Shortest-Job-First (SJF)

Ejemplo (sin desalojo)

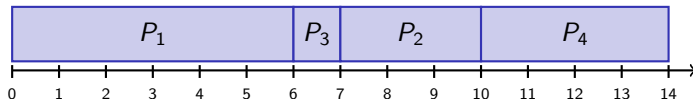
Misma instancia. En cada decisión elegimos, **entre los que ya llegaron**, el de ráfaga más corta. Sin desalojo: el que agarra la CPU la usa hasta terminar.



Shortest-Job-First (SJF)

Ejemplo (sin desalojo)

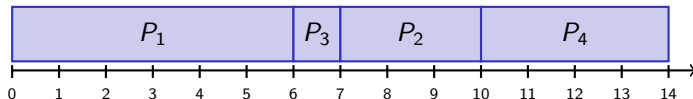
Misma instancia. En cada decisión elegimos, **entre los que ya llegaron**, el de ráfaga más corta. Sin desalojo: el que agarra la CPU la usa hasta terminar.



Shortest-Job-First (SJF)

Ejemplo (sin desalojo)

Misma instancia. En cada decisión elegimos, **entre los que ya llegaron**, el de ráfaga más corta. Sin desalojo: el que agarra la CPU la usa hasta terminar.

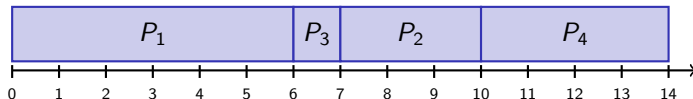


En $t = 0$ el único que llegó es P_1 , así que **no hay opción**: arranca P_1 y, como no hay desalojo, se queda hasta $t = 6$. Recién ahí elegimos entre $P_2(3)$, $P_3(1)$ y $P_4(4)$.

Shortest-Job-First (SJF)

Ejemplo (sin desalojo)

Misma instancia. En cada decisión elegimos, **entre los que ya llegaron**, el de ráfaga más corta. Sin desalojo: el que agarra la CPU la usa hasta terminar.



En $t = 0$ el único que llegó es P_1 , así que **no hay opción**: arranca P_1 y, como no hay desalojo, se queda hasta $t = 6$. Recién ahí elegimos entre $P_2(3)$, $P_3(1)$ y $P_4(4)$.

Turnaround promedio = $31/4 = 7,75$

Waiting time promedio = $17/4 = 4,25$

Shortest-Job-First (SJF)

Para pensar

- ¿Qué pasa si llega un proceso cuya próxima ráfaga de CPU es menor a lo que le falta al proceso ejecutando en ese momento?

Shortest-Job-First (SJF)

Para pensar

- ¿Qué pasa si llega un proceso cuya próxima ráfaga de CPU es menor a lo que le falta al proceso ejecutando en ese momento? → Depende.

Shortest-Job-First (SJF)

Para pensar

- ¿Qué pasa si llega un proceso cuya próxima ráfaga de CPU es menor a lo que le falta al proceso ejecutando en ese momento? → Depende.
 - Versión *nonpreemptive*: el proceso en ejecución continúa hasta que termina su ráfaga de CPU.
 - Versión *preemptive*: el scheduler desaloja al proceso en ejecución y le da la CPU al nuevo ([shortest-remaining-time-first](#) o SRTF).

Shortest-Job-First (SJF)

Para pensar

- ¿Qué pasa si llega un proceso cuya próxima ráfaga de CPU es menor a lo que le falta al proceso ejecutando en ese momento? → Depende.
 - Versión *nonpreemptive*: el proceso en ejecución continúa hasta que termina su ráfaga de CPU.
 - Versión *preemptive*: el scheduler desaloja al proceso en ejecución y le da la CPU al nuevo ([shortest-remaining-time-first](#) o SRTF).
- SJF es óptimo respecto al waiting time promedio. ¿Por qué no se implementa en la práctica?

Shortest-Job-First (SJF)

Para pensar

- ¿Qué pasa si llega un proceso cuya próxima ráfaga de CPU es menor a lo que le falta al proceso ejecutando en ese momento? → Depende.
 - Versión *nonpreemptive*: el proceso en ejecución continúa hasta que termina su ráfaga de CPU.
 - Versión *preemptive*: el scheduler desaloja al proceso en ejecución y le da la CPU al nuevo ([shortest-remaining-time-first](#) o SRTF).
- SJF es óptimo respecto al waiting time promedio. ¿Por qué no se implementa en la práctica? → no se puede saber de antemano la longitud de la próxima ráfaga de CPU. Se puede aproximar, pero igual es difícil.

Shortest-Remaining-Time-First (SRTF)

Definición

- Es la versión [con desalojo](#) de SJF.

Shortest-Remaining-Time-First (SRTF)

Definición

- Es la versión **con desalojo** de SJF.
- En lugar de comparar la ráfaga completa, se compara el **tiempo que le falta** a cada proceso (*remaining time*).

Shortest-Remaining-Time-First (SRTF)

Definición

- Es la versión **con desalojo** de SJF.
- En lugar de comparar la ráfaga completa, se compara el **tiempo que le falta** a cada proceso (*remaining time*).
- Cada vez que **llega** un proceso nuevo, el scheduler recalcula: si al recién llegado le falta menos que al que está corriendo, lo **desaloja**.

Shortest-Remaining-Time-First (SRTF)

Definición

- Es la versión **con desalojo** de SJF.
- En lugar de comparar la ráfaga completa, se compara el **tiempo que le falta** a cada proceso (*remaining time*).
- Cada vez que **llega** un proceso nuevo, el scheduler recalcula: si al recién llegado le falta menos que al que está corriendo, lo **desaloja**.
- Como puede desalojar, un mismo proceso puede aparecer en **varios bloques** del diagrama de Gantt.

Comparemos las cuatro políticas

Misma instancia, mismos procesos

Política	Waiting time	Turnaround	Response	Tramos de CPU
FCFS	4,75	8,25	4,75	4
SJF	4,25	7,75	4,25	4
SRTF	2,75	6,25	0,50	6
RR ($q = 2$)	5,25	8,75	1,75	8

Comparemos las cuatro políticas

Misma instancia, mismos procesos

Política	Waiting time	Turnaround	Response	Tramos de CPU
FCFS	4,75	8,25	4,75	4
SJF	4,25	7,75	4,25	4
SRTF	2,75	6,25	0,50	6
RR ($q = 2$)	5,25	8,75	1,75	8

- **SRTF gana en todo** lo que medimos. ¿Y entonces por qué no lo usamos siempre?

Comparemos las cuatro políticas

Misma instancia, mismos procesos

Política	Waiting time	Turnaround	Response	Tramos de CPU
FCFS	4,75	8,25	4,75	4
SJF	4,25	7,75	4,25	4
SRTF	2,75	6,25	0,50	6
RR ($q = 2$)	5,25	8,75	1,75	8

- **SRTF gana en todo** lo que medimos. ¿Y entonces por qué no lo usamos siempre?
- Porque necesita saber **el futuro**: cuánto le falta a cada proceso. En un sistema real no se conoce.

Comparemos las cuatro políticas

Misma instancia, mismos procesos

Política	Waiting time	Turnaround	Response	Tramos de CPU
FCFS	4,75	8,25	4,75	4
SJF	4,25	7,75	4,25	4
SRTF	2,75	6,25	0,50	6
RR ($q = 2$)	5,25	8,75	1,75	8

- **SRTF gana en todo** lo que medimos. ¿Y entonces por qué no lo usamos siempre?
- Porque necesita saber **el futuro**: cuánto le falta a cada proceso. En un sistema real no se conoce.
- Y porque **castiga a los largos**: los deja esperando indefinidamente.

Comparemos las cuatro políticas

Misma instancia, mismos procesos

Política	Waiting time	Turnaround	Response	Tramos de CPU
FCFS	4,75	8,25	4,75	4
SJF	4,25	7,75	4,25	4
SRTF	2,75	6,25	0,50	6
RR ($q = 2$)	5,25	8,75	1,75	8

- **SRTF gana en todo** lo que medimos. ¿Y entonces por qué no lo usamos siempre?
- Porque necesita saber **el futuro**: cuánto le falta a cada proceso. En un sistema real no se conoce.
- Y porque **castiga a los largos**: los deja esperando indefinidamente.
- **RR** es el peor en turnaround, pero no necesita adivinar nada y le da a todos un turno acotado: comparado con FCFS, baja el response de 4,75 a 1,75. Por eso es la política de los sistemas **interactivos**.

Comparemos las cuatro políticas

Misma instancia, mismos procesos

Política	Waiting time	Turnaround	Response	Tramos de CPU
FCFS	4,75	8,25	4,75	4
SJF	4,25	7,75	4,25	4
SRTF	2,75	6,25	0,50	6
RR ($q = 2$)	5,25	8,75	1,75	8

- **SRTF gana en todo** lo que medimos. ¿Y entonces por qué no lo usamos siempre?
- Porque necesita saber **el futuro**: cuánto le falta a cada proceso. En un sistema real no se conoce.
- Y porque **castiga a los largos**: los deja esperando indefinidamente.
- **RR** es el peor en turnaround, pero no necesita adivinar nada y le da a todos un turno acotado: comparado con FCFS, baja el response de 4,75 a 1,75. Por eso es la política de los sistemas **interactivos**.
- El precio de RR se ve en la última columna: **8 tramos** en vez de 4. Cada tramo de más es un **cambio de contexto**, y no es gratis.

Prioridades

Definición

- Se asocia a cada proceso una *prioridad*, y se elige para ejecutar al proceso más prioritario en estado *ready*.

Prioridades

Definición

- Se asocia a cada proceso una **prioridad**, y se elige para ejecutar al **proceso más prioritario en estado *ready***.
- Si hay varios procesos con la misma prioridad, se ejecutan en orden FCFS.

Prioridades

Definición

- Se asocia a cada proceso una **prioridad**, y se elige para ejecutar al **proceso más prioritario en estado *ready***.
- Si hay varios procesos con la misma prioridad, se ejecutan en orden FCFS.
- SJF es un caso particular de esquema por prioridades

Prioridades

Definición

- Se asocia a cada proceso una **prioridad**, y se elige para ejecutar al **proceso más prioritario en estado *ready***.
- Si hay varios procesos con la misma prioridad, se ejecutan en orden FCFS.
- SJF es un caso particular de esquema por prioridades (la prioridad p es proporcional al tiempo de la siguiente ráfaga de CPU).

Prioridades

Definición

- Se asocia a cada proceso una **prioridad**, y se elige para ejecutar al **proceso más prioritario en estado *ready***.
- Si hay varios procesos con la misma prioridad, se ejecutan en orden FCFS.
- SJF es un caso particular de esquema por prioridades (la prioridad p es proporcional al tiempo de la siguiente ráfaga de CPU).
- Puede ser *preemptive* o *nonpreemptive*.

Prioridades

Definición

- Se asocia a cada proceso una **prioridad**, y se elige para ejecutar al **proceso más prioritario en estado *ready***.
- Si hay varios procesos con la misma prioridad, se ejecutan en orden FCFS.
- SJF es un caso particular de esquema por prioridades (la prioridad p es proporcional al tiempo de la siguiente ráfaga de CPU).
- Puede ser *preemptive* o *nonpreemptive*.
- ¿Una mayor prioridad se denota con un número más chico o uno más grande?

Prioridades

Definición

- Se asocia a cada proceso una **prioridad**, y se elige para ejecutar al **proceso más prioritario en estado *ready***.
- Si hay varios procesos con la misma prioridad, se ejecutan en orden FCFS.
- SJF es un caso particular de esquema por prioridades (la prioridad p es proporcional al tiempo de la siguiente ráfaga de CPU).
- Puede ser *preemptive* o *nonpreemptive*.
- ¿Una mayor prioridad se denota con un número más chico o uno más grande? Depende de la documentación de se lea. En la práctica vamos a considerar el valor 0 como la mayor prioridad.

Prioridades

Ejemplo

Considerar los siguientes procesos, con el largo de la ráfaga de CPU indicada en milisegundos:

Proceso	Ráfaga de CPU	Prioridad
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Prioridades

Ejemplo

Considerar los siguientes procesos, con el largo de la ráfaga de CPU indicada en milisegundos:

Proceso	Ráfaga de CPU	Prioridad
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Diagrama de Gantt:



- Waiting time promedio: $(6 + 0 + 16 + 18 + 1)/5 = 8,2$ milisegundos
- Turnaround promedio: $(16 + 1 + 18 + 19 + 6)/5 = 12$ milisegundos

Prioridades

Para pensar

- ¿Qué pasa si constantemente llegan procesos de alta prioridad?

Prioridades

Para pensar

- ¿Qué pasa si constantemente llegan procesos de alta prioridad? → *Starvation*.

Prioridades

Para pensar

- ¿Qué pasa si constantemente llegan procesos de alta prioridad? → *Starvation*.
- ¿Hay formas de prevenir esto?

Prioridades

Para pensar

- ¿Qué pasa si constantemente llegan procesos de alta prioridad? → *Starvation*.
- ¿Hay formas de prevenir esto? → Sí:
 - **Aging:** gradualmente incrementar la prioridad de los procesos que están esperando hace mucho.

Prioridades

Para pensar

- ¿Qué pasa si constantemente llegan procesos de alta prioridad? → *Starvation*.
- ¿Hay formas de prevenir esto? → Sí:
 - **Aging:** gradualmente incrementar la prioridad de los procesos que están esperando hace mucho.
- ¿Cuánto cuesta buscar en la cola *ready* el proceso con mayor prioridad? (asumiendo que la cola esta modelada con una lista)

Prioridades

Para pensar

- ¿Qué pasa si constantemente llegan procesos de alta prioridad? → *Starvation*.
- ¿Hay formas de prevenir esto? → Sí:
 - **Aging:** gradualmente incrementar la prioridad de los procesos que están esperando hace mucho.
- ¿Cuánto cuesta buscar en la cola *ready* el proceso con mayor prioridad? (asumiendo que la cola esta modelada con una lista) → $O(n)$.

Prioridades

Para pensar

- ¿Qué pasa si constantemente llegan procesos de alta prioridad? → *Starvation*.
- ¿Hay formas de prevenir esto? → Sí:
 - **Aging:** gradualmente incrementar la prioridad de los procesos que están esperando hace mucho.
- ¿Cuánto cuesta buscar en la cola *ready* el proceso con mayor prioridad? (asumiendo que la cola esta modelada con una lista) → $O(n)$.
- ¿Y si **dos procesos empatan** en prioridad?

Prioridades

Para pensar

- ¿Qué pasa si constantemente llegan procesos de alta prioridad? → *Starvation*.
- ¿Hay formas de prevenir esto? → Sí:
 - **Aging:** gradualmente incrementar la prioridad de los procesos que están esperando hace mucho.
- ¿Cuánto cuesta buscar en la cola *ready* el proceso con mayor prioridad? (asumiendo que la cola esta modelada con una lista) → $O(n)$.
- ¿Y si **dos procesos empatan** en prioridad? → se desempata por **orden de llegada** (FCFS dentro de la misma prioridad).

Multilevel Queue

Definición

- Se mantienen **colas separadas para cada prioridad**.

Multilevel Queue

Definición

- Se mantienen **colas separadas para cada prioridad**.
- Se ejecutan primero los procesos en la **cola de mayor prioridad**.

Multilevel Queue

Definición

- Se mantienen **colas separadas para cada prioridad**.
- Se ejecutan primero los procesos en la **cola de mayor prioridad**.
- Cada cola tiene **prioridad absoluta** sobre las colas de menor prioridad.

Multilevel Queue

Definición

- Se mantienen **colas separadas para cada prioridad**.
- Se ejecutan primero los procesos en la **cola de mayor prioridad**.
- Cada cola tiene **prioridad absoluta** sobre las colas de menor prioridad.
- La prioridad de cada proceso es **estática**, así que cada proceso vive siempre en la misma cola.

Multilevel Queue

Definición

- Se mantienen **colas separadas para cada prioridad**.
- Se ejecutan primero los procesos en la **cola de mayor prioridad**.
- Cada cola tiene **prioridad absoluta** sobre las colas de menor prioridad.
- La prioridad de cada proceso es **estática**, así que cada proceso vive siempre en la misma cola.
- Se suele usar RR dentro de cada cola, aunque puede variar.

Multilevel Queue

Definición

- Se mantienen **colas separadas para cada prioridad**.
- Se ejecutan primero los procesos en la **cola de mayor prioridad**.
- Cada cola tiene **prioridad absoluta** sobre las colas de menor prioridad.
- La prioridad de cada proceso es **estática**, así que cada proceso vive siempre en la misma cola.
- Se suele usar RR dentro de cada cola, aunque puede variar.
- Se suele aplicar para **particionar los procesos** que requieren distinto *response-time*.

Multilevel Queue

Definición

- Se mantienen **colas separadas para cada prioridad**.
- Se ejecutan primero los procesos en la **cola de mayor prioridad**.
- Cada cola tiene **prioridad absoluta** sobre las colas de menor prioridad.
- La prioridad de cada proceso es **estática**, así que cada proceso vive siempre en la misma cola.
- Se suele usar RR dentro de cada cola, aunque puede variar.
- Se suele aplicar para **particionar los procesos** que requieren distinto *response-time*.

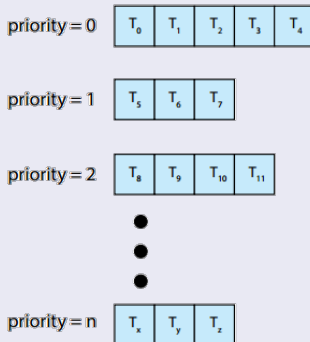


Figura: Colas separadas para cada prioridad³

Multilevel Queue

Para pensar

Una forma de particionar procesos

- 1 **Real-time:** procesos con restricciones de tiempo fijas y bien definidas, deben devolver el resultado correcto dentro de sus limitaciones de tiempo (ej: sistemas de imágenes médicas, sistemas de control industrial, etc.).

Multilevel Queue

Para pensar

Una forma de particionar procesos

- 1 **Real-time:** procesos con restricciones de tiempo fijas y bien definidas, deben devolver el resultado correcto dentro de sus limitaciones de tiempo (ej: sistemas de imágenes médicas, sistemas de control industrial, etc.).
- 2 **Procesos interactivos:** programas de propósito general que se ejecutan tomando *input* por parte del usuario, quien espera una respuesta rápida (ej: procesadores de texto, planillas de cálculo, etc.).

Multilevel Queue

Para pensar

Una forma de particionar procesos

- 1 **Real-time:** procesos con restricciones de tiempo fijas y bien definidas, deben devolver el resultado correcto dentro de sus limitaciones de tiempo (ej: sistemas de imágenes médicas, sistemas de control industrial, etc.).
- 2 **Procesos interactivos:** programas de propósito general que se ejecutan tomando *input* por parte del usuario, quien espera una respuesta rápida (ej: procesadores de texto, planillas de cálculo, etc.).
- 3 **Procesos batch (lotes):** tareas periódicas y repetitivas con *input* predeterminado desde archivos u otra fuente de datos, generalmente de gran volumen (ej: copias de seguridad, recopilación de datos, facturaciones, generación de informes, etc.).

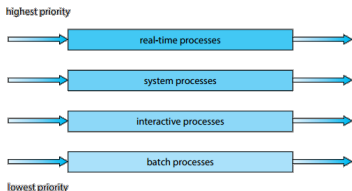
Multilevel Queue

Para pensar

Una forma de particionar procesos

- 1 **Real-time:** procesos con restricciones de tiempo fijas y bien definidas, deben devolver el resultado correcto dentro de sus limitaciones de tiempo (ej: sistemas de imágenes médicas, sistemas de control industrial, etc.).
- 2 **Procesos interactivos:** programas de propósito general que se ejecutan tomando *input* por parte del usuario, quien espera una respuesta rápida (ej: procesadores de texto, planillas de cálculo, etc.).
- 3 **Procesos batch (lotes):** tareas periódicas y repetitivas con *input* predeterminado desde archivos u otra fuente de datos, generalmente de gran volumen (ej: copias de seguridad, recopilación de datos, facturaciones, generación de informes, etc.).

- ¿Qué tipo de procesos tendrán más prioridad? ¿Cuáles tendrán menos?

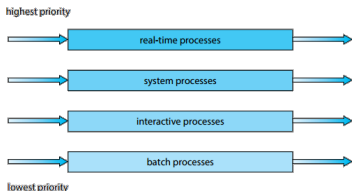


Multilevel Queue

Para pensar

Una forma de particionar procesos

- 1 **Real-time:** procesos con restricciones de tiempo fijas y bien definidas, deben devolver el resultado correcto dentro de sus limitaciones de tiempo (ej: sistemas de imágenes médicas, sistemas de control industrial, etc.).
- 2 **Procesos interactivos:** programas de propósito general que se ejecutan tomando *input* por parte del usuario, quien espera una respuesta rápida (ej: procesadores de texto, planillas de cálculo, etc.).
- 3 **Procesos batch (lotes):** tareas periódicas y repetitivas con *input* predeterminado desde archivos u otra fuente de datos, generalmente de gran volumen (ej: copias de seguridad, recopilación de datos, facturaciones, generación de informes, etc.).



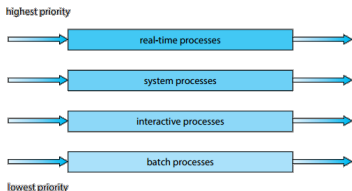
- ¿Qué tipo de procesos tendrán más prioridad? ¿Cuáles tendrán menos?
- ¿Qué pasa si constantemente llegan procesos a la cola más prioritaria?

Multilevel Queue

Para pensar

Una forma de particionar procesos

- 1 **Real-time:** procesos con restricciones de tiempo fijas y bien definidas, deben devolver el resultado correcto dentro de sus limitaciones de tiempo (ej: sistemas de imágenes médicas, sistemas de control industrial, etc.).
- 2 **Procesos interactivos:** programas de propósito general que se ejecutan tomando *input* por parte del usuario, quien espera una respuesta rápida (ej: procesadores de texto, planillas de cálculo, etc.).
- 3 **Procesos batch (lotes):** tareas periódicas y repetitivas con *input* predeterminado desde archivos u otra fuente de datos, generalmente de gran volumen (ej: copias de seguridad, recopilación de datos, facturaciones, generación de informes, etc.).



- ¿Qué tipo de procesos tendrán más prioridad? ¿Cuáles tendrán menos?
- ¿Qué pasa si constantemente llegan procesos a la cola más prioritaria? → *Starvation*.

Multilevel Feedback Queue

Definición

- Se permite a los procesos cambiarse de cola.

Multilevel Feedback Queue

Definición

- Se permite a los procesos **cambiarse de cola**.
- Suele separar a los procesos según sus ráfagas de CPU: a mayor uso de CPU, más baja la prioridad.

Multilevel Feedback Queue

Definición

- Se permite a los procesos **cambiarse de cola**.
- Suele separar a los procesos según sus ráfagas de CPU: a mayor uso de CPU, más baja la prioridad.
- Puede implementar *aging* para evitar *starvation*.

Para pensar: ¿Qué tipos de procesos terminan quedando con mayor prioridad?

Multilevel Feedback Queue

Definición

- Se permite a los procesos **cambiarse de cola**.
- Suele separar a los procesos según sus ráfagas de CPU: a mayor uso de CPU, más baja la prioridad.
- Puede implementar *aging* para evitar *starvation*.

Para pensar: ¿Qué tipos de procesos terminan quedando con mayor prioridad? → intensivos en I/O e interactivos (generalmente tienen ráfagas de CPU más cortas).

Multilevel Feedback Queue

Tener en cuenta al diseñar un *Multilevel Feedback Queue Scheduling*:

Multilevel Feedback Queue

Tener en cuenta al diseñar un *Multilevel Feedback Queue Scheduling*:

- Cantidad de colas.

Multilevel Feedback Queue

Tener en cuenta al diseñar un *Multilevel Feedback Queue Scheduling*:

- Cantidad de colas.
- Algoritmo de scheduling en cada cola.

Multilevel Feedback Queue

Tener en cuenta al diseñar un *Multilevel Feedback Queue Scheduling*:

- Cantidad de colas.
- Algoritmo de scheduling en cada cola.
- Por qué un proceso podría pasar a una cola de mayor prioridad.

Multilevel Feedback Queue

Tener en cuenta al diseñar un *Multilevel Feedback Queue Scheduling*:

- Cantidad de colas.
- Algoritmo de scheduling en cada cola.
- Por qué un proceso podría pasar a una cola de mayor prioridad.
- Por qué un proceso podría pasar a una cola de menor prioridad.

Multilevel Feedback Queue

Tener en cuenta al diseñar un *Multilevel Feedback Queue Scheduling*:

- Cantidad de colas.
- Algoritmo de scheduling en cada cola.
- Por qué un proceso podría pasar a una cola de mayor prioridad.
- Por qué un proceso podría pasar a una cola de menor prioridad.
- Cómo elijo a qué cola va un proceso cuando recién llega.
- ¿Puede haber *starvation*? ¿Por qué? ¿Puedo/quiero evitarlo? ¿Cómo?

Momento para preguntas

Menú para hoy

- 1 Repaso
- 2 Conceptos básicos
- 3 Criterios y objetivos
- 4 Algoritmos de scheduling
- 5 Ejercicio de diseño**
- 6 Scheduling de Tiempo Real
- 7 El simulador del TP
- 8 Scheduling en la vida real
- 9 Cierre

Ejercicio de diseño

Enunciado

Un sistema que atiende tareas **interactivas** de varias sucursales bancarias está conectado en forma directa a la central policial, y frente a un caso de robo, genera un proceso que activa la alarma central.

Diseñar un algoritmo de scheduling que permita que, una vez generado ese proceso de alarma, tenga prioridad sobre el resto de las tareas. Tener en cuenta que pueden generarse distintas alarmas desde distintas sucursales.

Ejercicio de diseño

Analicemos el enunciado

- Hay tareas **interactivas**.

Ejercicio de diseño

Analicemos el enunciado

- Hay tareas **interactivas**.
- Eventualmente se lanzan **alarmas**.

Ejercicio de diseño

Analicemos el enunciado

- Hay tareas **interactivas**.
 - Tienden a ser intensivas en I/O (ráfagas cortas de CPU).
 - Deben tener buen tiempo de respuesta.
 - Son potencialmente muchas.
- Eventualmente se lanzan **alarmas**.

Ejercicio de diseño

Analicemos el enunciado

- Hay tareas **interactivas**.
 - Tienden a ser intensivas en I/O (ráfagas cortas de CPU).
 - Deben tener buen tiempo de respuesta.
 - Son potencialmente muchas.
- Eventualmente se lanzan **alarmas**.
 - Deben priorizarse sobre las otras tareas.
 - Pueden suceder varias alarmas en simultáneo.

Ejercicio de diseño

Analicemos el enunciado

- Hay tareas **interactivas**. → **RR**
 - Tienden a ser intensivas en I/O (ráfagas cortas de CPU).
 - Deben tener buen tiempo de respuesta.
 - Son potencialmente muchas.
- Eventualmente se lanzan **alarmas**.
 - Deben priorizarse sobre las otras tareas.
 - Pueden suceder varias alarmas en simultáneo.

Ejercicio de diseño

Analicemos el enunciado

- Hay tareas **interactivas**. → **RR**
 - Tienden a ser intensivas en I/O (ráfagas cortas de CPU).
 - Deben tener buen tiempo de respuesta.
 - Son potencialmente muchas.
- Eventualmente se lanzan **alarmas**.
 - Deben priorizarse sobre las otras tareas. → **Prioridades**
 - Pueden suceder varias alarmas en simultáneo.

Ejercicio de diseño

Analicemos el enunciado

- Hay tareas **interactivas**. → **RR**
 - Tienden a ser intensivas en I/O (ráfagas cortas de CPU).
 - Deben tener buen tiempo de respuesta.
 - Son potencialmente muchas.
- Eventualmente se lanzan **alarmas**.
 - Deben priorizarse sobre las otras tareas. → **Prioridades**
 - Pueden suceder varias alarmas en simultáneo. → **Separar alarmas de otros procesos**

Ejercicio de diseño

Analicemos el enunciado

- Hay tareas **interactivas**. → **RR**
 - Tienden a ser intensivas en I/O (ráfagas cortas de CPU).
 - Deben tener buen tiempo de respuesta.
 - Son potencialmente muchas.
- Eventualmente se lanzan **alarmas**. → **Priority Queue**
 - Deben priorizarse sobre las otras tareas. → **Prioridades**
 - Pueden suceder varias alarmas en simultáneo. → **Separar alarmas de otros procesos**

Ejercicio de diseño

Posible solución

- Proponemos un scheduler que funcione con dos colas de prioridades estáticas.

Ejercicio de diseño

Posible solución

- Proponemos un scheduler que funcione con **dos colas de prioridades estáticas**.
- La cola **más prioritaria** se asignará a los procesos de **alarma**.
- La cola **menos prioritaria** se asignará a los procesos **interactivos**.

Ejercicio de diseño

Posible solución

- Proponemos un scheduler que funcione con **dos colas de prioridades estáticas**.
- La cola **más prioritaria** se asignará a los procesos de **alarma**.
 - Esta cola utilizará un esquema **FCFS**: atenderá las alarmas en orden de llegada en caso de haber más de una.
 - Contamos con que este tipo de proceso es corto: enciende la alarma y termina.
- La cola **menos prioritaria** se asignará a los procesos **interactivos**.

Ejercicio de diseño

Posible solución

- Proponemos un scheduler que funcione con **dos colas de prioridades estáticas**.
- La cola **más prioritaria** se asignará a los procesos de **alarma**.
 - Esta cola utilizará un esquema **FCFS**: atenderá las alarmas en orden de llegada en caso de haber más de una.
 - Contamos con que este tipo de proceso es corto: enciende la alarma y termina.
- La cola **menos prioritaria** se asignará a los procesos **interactivos**.
 - Esta cola utilizará un esquema **RR**: se asignará un quantum razonable a cada proceso.
 - Además, al ser intensivos en I/O, se bloquearán seguido, permitiendo ejecutar otros procesos en la misma cola.

Ejercicio de diseño

Posible solución

- Posibilidades de *starvation*:

Ejercicio de diseño

Posible solución

- Posibilidades de *starvation*:
 - Entre los procesos interactivos no puede haber *starvation*: todos ejecutarán un quantum a su tiempo.

- Posibilidades de *starvation*:
 - Entre los procesos interactivos no puede haber *starvation*: todos ejecutarán un quantum a su tiempo.
 - Entre los procesos de alarma no puede haber *starvation*: aunque lleguen constantemente alarmas, se resolverán rápido y permitirán pasar a las siguientes.

Ejercicio de diseño

Posible solución

- Posibilidades de *starvation*:

- Entre los procesos interactivos no puede haber *starvation*: todos ejecutarán un quantum a su tiempo.
- Entre los procesos de alarma no puede haber *starvation*: aunque lleguen constantemente alarmas, se resolverán rápido y permitirán pasar a las siguientes.
- Entre colas **podría existir *starvation* en caso de llegar constantes alarmas.**

Ejercicio de diseño

Posible solución

- Posibilidades de *starvation*:

- Entre los procesos interactivos no puede haber *starvation*: todos ejecutarán un quantum a su tiempo.
- Entre los procesos de alarma no puede haber *starvation*: aunque lleguen constantemente alarmas, se resolverán rápido y permitirán pasar a las siguientes.
- Entre colas **podría existir *starvation* en caso de llegar constantes alarmas.**
 - Si llegan múltiples alarmas, están robando en muchas sucursales, y pocos usuarios estarán utilizando el sistema interactivo. **No me preocupo por salvar estos procesos.**

Ejercicio de diseño

Para pensar

¿Cómo cambia el diseño si...

- ...hay que garantizar el funcionamiento de los procesos interactivos aunque haya múltiples alarmas?

Ejercicio de diseño

Para pensar

¿Cómo cambia el diseño si...

- ...hay que garantizar el funcionamiento de los procesos interactivos aunque haya múltiples alarmas?
- ...también hay procesos nocturnos que realizan backups?

Ejercicio de diseño

Para pensar

¿Cómo cambia el diseño si...

- ...hay que garantizar el funcionamiento de los procesos interactivos aunque haya múltiples alarmas?
- ...también hay procesos nocturnos que realizan backups?
- ...los procesos interactivos pueden eventualmente solicitar reportes que demandan mayor procesamiento?

Menú para hoy

- 1 Repaso
- 2 Conceptos básicos
- 3 Criterios y objetivos
- 4 Algoritmos de scheduling
- 5 Ejercicio de diseño
- 6 Scheduling de Tiempo Real**
- 7 El simulador del TP
- 8 Scheduling en la vida real
- 9 Cierre

Scheduling de Tiempo Real

Definición

- El scheduling de CPU para sistemas operativos de tiempo real tiene sus propios problemas.

Scheduling de Tiempo Real

Definición

- El scheduling de CPU para sistemas operativos de tiempo real tiene sus propios problemas.
- Podemos clasificarlos en:
 - Sistemas **soft real-time**
 - Sistemas **hard real-time**

Scheduling de Tiempo Real

Definición

- El scheduling de CPU para sistemas operativos de tiempo real tiene sus propios problemas.
- Podemos clasificarlos en:
 - Sistemas **soft real-time**
 - No dan garantías sobre cuándo se va a poner a ejecutar un proceso RT.
 - Sólo garantizan que los procesos van a ejecutar con más prioridad que los procesos no críticos.
 - Sistemas **hard real-time**

Scheduling de Tiempo Real

Definición

- El scheduling de CPU para sistemas operativos de tiempo real tiene sus propios problemas.
- Podemos clasificarlos en:
 - Sistemas **soft real-time**
 - No dan garantías sobre cuándo se va a poner a ejecutar un proceso RT.
 - Sólo garantizan que los procesos van a ejecutar con más prioridad que los procesos no críticos.
 - Sistemas **hard real-time**
 - Tienen requerimientos más estrictos: una tarea crítica debe ser ejecutada dentro de un *deadline*.
 - Ejecutar con *deadline* vencido es lo mismo que no haber ejecutado.

Scheduling de Tiempo Real

Minimizar la latencia

- Un sistema RT típicamente está esperando que ocurra un evento RT.

Scheduling de Tiempo Real

Minimizar la latencia

- Un sistema RT típicamente está esperando que ocurra un evento RT.
- Los eventos pueden surgir desde el software (ej: un timer) o desde el hardware (ej: señal desde un sensor).

Scheduling de Tiempo Real

Minimizar la latencia

- Un sistema RT típicamente está esperando que ocurra un evento RT.
- Los eventos pueden surgir desde el software (ej: un timer) o desde el hardware (ej: señal desde un sensor).
- Cuando ocurre un evento, el sistema debe responder lo más rápido posible.

Scheduling de Tiempo Real

Minimizar la latencia

- Un sistema RT típicamente está esperando que ocurra un evento RT.
- Los eventos pueden surgir desde el software (ej: un timer) o desde el hardware (ej: señal desde un sensor).
- Cuando ocurre un evento, el sistema debe responder lo más rápido posible.

El tiempo que pasa entre que ocurre un evento hasta que el proceso RT se ejecuta es llamado **event latency**.

Scheduling de Tiempo Real

Minimizar la latencia

- Un sistema RT típicamente está esperando que ocurra un evento RT.
- Los eventos pueden surgir desde el software (ej: un timer) o desde el hardware (ej: señal desde un sensor).
- Cuando ocurre un evento, el sistema debe responder lo más rápido posible.

El tiempo que pasa entre que ocurre un evento hasta que el proceso RT se ejecuta es llamado **event latency**.

- Algunos sistemas pueden tener requerimientos de latencia de pocos milisegundos, como el sistema que controla los frenos de un auto.

Scheduling de Tiempo Real

Minimizar la latencia

- Un sistema RT típicamente está esperando que ocurra un evento RT.
- Los eventos pueden surgir desde el software (ej: un timer) o desde el hardware (ej: señal desde un sensor).
- Cuando ocurre un evento, el sistema debe responder lo más rápido posible.

El tiempo que pasa entre que ocurre un evento hasta que el proceso RT se ejecuta es llamado **event latency**.

- Algunos sistemas pueden tener requerimientos de latencia de pocos milisegundos, como el sistema que controla los frenos de un auto.
- Otros sistemas pueden tolerar varios segundos de latencia, como el sistema de control de un radar.

Scheduling de Tiempo Real

Algoritmos de Scheduling

Prioridades

- La característica más importante de sistemas operativos RT es que tienen que poder responder a un proceso RT que requiere la CPU tan pronto se pueda.

Scheduling de Tiempo Real

Algoritmos de Scheduling

Prioridades

- La característica más importante de sistemas operativos RT es que tienen que poder responder a un proceso RT que requiere la CPU tan pronto se pueda.
- Por lo tanto, un sistema RT debe soportar un algoritmo de scheduling basado en *prioridades con preemption*.

Earliest-Deadline-First (EDF)

- EDF *asigna prioridades dinámicamente según el **deadline***. Cuanto más pronto el *deadline*, más prioridad.

Scheduling de Tiempo Real

Algoritmos de Scheduling

Prioridades

- La característica más importante de sistemas operativos RT es que tienen que poder responder a un proceso RT que requiere la CPU tan pronto se pueda.
- Por lo tanto, un sistema RT debe soportar un algoritmo de scheduling basado en *prioridades con preemption*.

Earliest-Deadline-First (EDF)

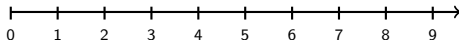
- EDF *asigna prioridades dinámicamente según el *deadline**. Cuanto más pronto el *deadline*, más prioridad.
- Cuando un proceso puede ejecutar, debe anunciar sus requerimientos de *deadline* al sistema. Las prioridades deben ajustarse para reflejar el *deadline* del nuevo proceso.
- En teoría, EDF es óptimo ya que cada proceso puede cumplir su *deadline* y se maximiza el uso de CPU. En la práctica, no siempre es factible.

Earliest-Deadline-First (EDF)

Ejemplo con números

EDF acá es **con desalojo**: cada vez que llega una tarea, se compara su *deadline* con el de la que está corriendo.

	Lleg.	Ejec.	<i>Dline</i>
T_1	0	4	9
T_2	1	2	4
T_3	3	3	12

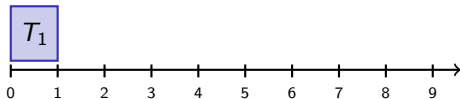


Earliest-Deadline-First (EDF)

Ejemplo con números

EDF acá es **con desalojo**: cada vez que llega una tarea, se compara su *deadline* con el de la que está corriendo.

	Lleg.	Ejec.	<i>Dline</i>
T_1	0	4	9
T_2	1	2	4
T_3	3	3	12



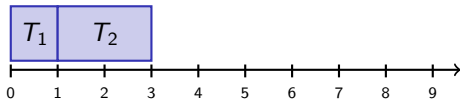
- $t = 0$: sólo está T_1 ($d = 9$), arranca.

Earliest-Deadline-First (EDF)

Ejemplo con números

EDF acá es **con desalojo**: cada vez que llega una tarea, se compara su *deadline* con el de la que está corriendo.

	Lleg.	Ejec.	Dline
T_1	0	4	9
T_2	1	2	4
T_3	3	3	12



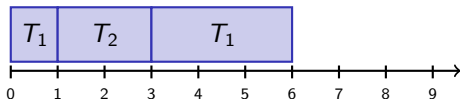
- $t = 0$: sólo está T_1 ($d = 9$), arranca.
- $t = 1$: llega T_2 con $d = 4 < 9 \rightarrow$ **desaloja** a T_1 .

Earliest-Deadline-First (EDF)

Ejemplo con números

EDF acá es **con desalojo**: cada vez que llega una tarea, se compara su *deadline* con el de la que está corriendo.

	Lleg.	Ejec.	<i>Dline</i>
T_1	0	4	9
T_2	1	2	4
T_3	3	3	12



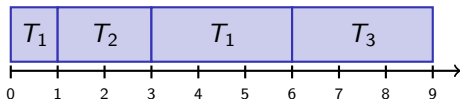
- $t = 0$: sólo está T_1 ($d = 9$), arranca.
- $t = 1$: llega T_2 con $d = 4 < 9 \rightarrow$ **desaloja** a T_1 .
- $t = 3$: termina T_2 y llega T_3 ($d = 12$). Entre T_1 ($d = 9$) y T_3 ($d = 12$) gana T_1 .

Earliest-Deadline-First (EDF)

Ejemplo con números

EDF acá es **con desalojo**: cada vez que llega una tarea, se compara su *deadline* con el de la que está corriendo.

	Lleg.	Ejec.	Dline
T_1	0	4	9
T_2	1	2	4
T_3	3	3	12



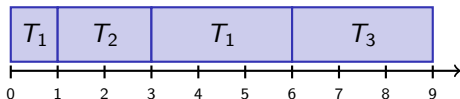
- $t = 0$: sólo está T_1 ($d = 9$), arranca.
- $t = 1$: llega T_2 con $d = 4 < 9 \rightarrow$ **desaloja** a T_1 .
- $t = 3$: termina T_2 y llega T_3 ($d = 12$). Entre T_1 ($d = 9$) y T_3 ($d = 12$) gana T_1 .

Earliest-Deadline-First (EDF)

Ejemplo con números

EDF acá es **con desalojo**: cada vez que llega una tarea, se compara su *deadline* con el de la que está corriendo.

	Lleg.	Ejec.	Dline
T_1	0	4	9
T_2	1	2	4
T_3	3	3	12

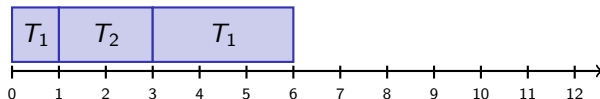


- $t = 0$: sólo está T_1 ($d = 9$), arranca.
- $t = 1$: llega T_2 con $d = 4 < 9 \rightarrow$ **desaloja** a T_1 .
- $t = 3$: termina T_2 y llega T_3 ($d = 12$). Entre T_1 ($d = 9$) y T_3 ($d = 12$) gana T_1 .
- Los tres cumplen su *deadline*. Turnaround promedio = $14/3 \approx 4,67$; waiting time promedio = $5/3 \approx 1,67$.

Earliest-Deadline-First (EDF)

EDF no hace magia

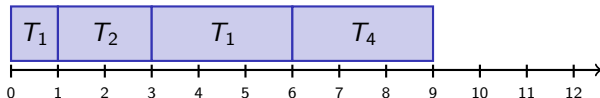
¿Qué pasa si además llega $T_4 = (\text{llegada } 6, \text{ ejecución } 3, \text{ deadline } 8)$?



Earliest-Deadline-First (EDF)

EDF no hace magia

¿Qué pasa si además llega $T_4 =$ (llegada 6, ejecución 3, *deadline* 8)?

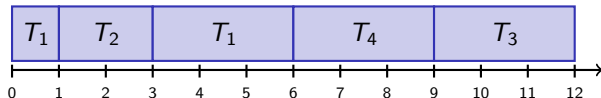


- T_4 llega con el *deadline* **más temprano** de todos, así que EDF le da la CPU **inmediatamente**.

Earliest-Deadline-First (EDF)

EDF no hace magia

¿Qué pasa si además llega $T_4 = (\text{llegada } 6, \text{ ejecución } 3, \text{ deadline } 8)$?

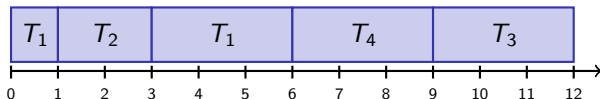


- T_4 llega con el *deadline* **más temprano** de todos, así que EDF le da la CPU **inmediatamente**.
- Y aun así **pierde su deadline**: necesita 3 unidades y sólo tiene 2 antes de $t = 8$.

Earliest-Deadline-First (EDF)

EDF no hace magia

¿Qué pasa si además llega $T_4 = (\text{llegada } 6, \text{ ejecución } 3, \text{ deadline } 8)$?



- T_4 llega con el *deadline* **más temprano** de todos, así que EDF le da la CPU **inmediatamente**.
- Y aun así **pierde su deadline**: necesita 3 unidades y sólo tiene 2 antes de $t = 8$.

EDF hizo todo bien: el problema es que la instancia era **infactible**. Ninguna política puede planificar lo que no entra en la CPU. Por eso los sistemas hard RT hacen un **test de planificabilidad** **antes** de aceptar una tarea.

Scheduling de Tiempo Real

Para pensar

- En el ejemplo dado, ¿qué tipo de proceso es la activación de alarma?
- ¿Cambia en algo la solución propuesta?

Menú para hoy

- 1 Repaso
- 2 Conceptos básicos
- 3 Criterios y objetivos
- 4 Algoritmos de scheduling
- 5 Ejercicio de diseño
- 6 Scheduling de Tiempo Real
- 7 El simulador del TP**
- 8 Scheduling en la vida real
- 9 Cierre

El simulador de MLFQ

La herramienta del TP1

El **TP1** usa `mlfq.py`, el simulador de MLFQ del libro [OSTEP](#).

- Un trabajo se describe con [tres](#) números: `llegada,duración,frecuencia_de_E/S`.

El simulador de MLFQ

La herramienta del TP1

El **TP1** usa `mlfq.py`, el simulador de MLFQ del libro [OSTEP](#).

- Un trabajo se describe con tres números: llegada, duración, frecuencia de E/S.
- `./mlfq.py -l 0,10,0:6,4,0 -Q 2,4,8 -A 1,1,1 -c`

El simulador de MLFQ

La herramienta del TP1

El **TP1** usa `mlfq.py`, el simulador de MLFQ del libro [OSTEP](#).

- Un trabajo se describe con [tres](#) números: llegada, duración, frecuencia de E/S.
- `./mlfq.py -l 0,10,0:6,4,0 -Q 2,4,8 -A 1,1,1 -c`
- ... que es **la misma instancia que hicimos a mano** hace un rato. La traza tiene que dar igual.

El simulador de MLFQ

La herramienta del TP1

El **TP1** usa `mlfq.py`, el simulador de MLFQ del libro [OSTEP](#).

- Un trabajo se describe con [tres](#) números: `llegada,duración,frecuencia_de_E/S`.
- `./mlfq.py -l 0,10,0:6,4,0 -Q 2,4,8 -A 1,1,1 -c`
- ... que es **la misma instancia que hicimos a mano** hace un rato. La traza tiene que dar igual.
- En la salida, `PRIORITY` es el nivel de cola: es el eje vertical de nuestro diagrama.

El simulador de MLFQ

La herramienta del TP1

El **TP1** usa `mlfq.py`, el simulador de MLFQ del libro [OSTEP](#).

- Un trabajo se describe con [tres](#) números: `llegada,duración,frecuencia_de_E/S`.
- `./mlfq.py -l 0,10,0:6,4,0 -Q 2,4,8 -A 1,1,1 -c`
- ... que es **la misma instancia que hicimos a mano** hace un rato. La traza tiene que dar igual.
- En la salida, `PRIORITY` es el nivel de cola: es el eje vertical de nuestro diagrama.

Las opciones que importan

`-Q` *quantum* por nivel `-A` *allotment* por nivel `-B` cada cuánto el *boost*
`-i` duración de la E/S `-S` el que hace E/S **no** baja de nivel (la regla vulnerable)
`-I` el que termina la E/S va al [frente](#) de la cola `-c` calcular métricas

El simulador de MLFQ

Qué van a tener que hacer en el TP

- 1 Generar problemas al azar y explicar los resultados ($-j$, $-s$) $\rightarrow$ hoy lo corrimos.

El simulador de MLFQ

Qué van a tener que hacer en el TP

- 1 Generar problemas al azar y explicar los resultados ($-j$, $-s$) $\rightarrow$ hoy lo corrimos.
- 2 **Extender el simulador** para que calcule **throughput** y **waiting time**. Ojo: hoy mlfq.py da **response** y **turnaround**, pero **no** waiting time.

El simulador de MLFQ

Qué van a tener que hacer en el TP

- 1 Generar problemas al azar y explicar los resultados (-j, -s) → hoy lo corrimos.
- 2 **Extender el simulador** para que calcule **throughput** y **waiting time**. Ojo: hoy mlfq.py da **response** y **turnaround**, pero **no** waiting time.
- 3 Mostrar un proceso **malicioso** que acapara la CPU con E/S → es el -S que acabamos de ver.

El simulador de MLFQ

Qué van a tener que hacer en el TP

- 1 Generar problemas al azar y explicar los resultados (-j, -s) → hoy lo corrimos.
- 2 **Extender el simulador** para que calcule **throughput** y **waiting time**. Ojo: hoy mlfq.py da **response** y **turnaround**, pero **no** waiting time.
- 3 Mostrar un proceso **malicioso** que acapara la CPU con E/S → es el -S que acabamos de ver.
- 4 Encontrar dos configuraciones opuestas para una carga mixta: una que favorezca al interactivo y otra que lo perjudique.

El simulador de MLFQ

Qué van a tener que hacer en el TP

- 1 Generar problemas al azar y explicar los resultados ($-j$, $-s$) $\rightarrow$ hoy lo corrimos.
- 2 **Extender el simulador** para que calcule **throughput** y **waiting time**. Ojo: hoy mlfq.py da **response** y **turnaround**, pero **no** waiting time.
- 3 Mostrar un proceso **malicioso** que acapara la CPU con E/S $\rightarrow$ es el $-S$ que acabamos de ver.
- 4 Encontrar dos configuraciones opuestas para una carga mixta: una que favorezca al interactivo y otra que lo perjudique.

Todo lo de esta clase apunta ahí: si entienden por qué B terminó antes que A en nuestro diagrama, el TP es jugar con los parámetros y medir.

Menú para hoy

- 1 Repaso
- 2 Conceptos básicos
- 3 Criterios y objetivos
- 4 Algoritmos de scheduling
- 5 Ejercicio de diseño
- 6 Scheduling de Tiempo Real
- 7 El simulador del TP
- 8 Scheduling en la vida real**
- 9 Cierre

¿Y qué usa Linux hoy?

- Durante 15 años, Linux usó **CFS** ([Completely Fair Scheduler](#)): la idea era repartir la CPU de forma proporcional, dándole el procesador al proceso que **menos** tiempo virtual acumuló.

¿Y qué usa Linux hoy?

- Durante 15 años, Linux usó **CFS** (**Completely Fair Scheduler**): la idea era repartir la CPU de forma proporcional, dándole el procesador al proceso que **menos** tiempo virtual acumuló.
- Desde **Linux 6.6 (2023)**, CFS fue **reemplazado** por **EEVDF**: **Earliest Eligible Virtual Deadline First**.

¿Y qué usa Linux hoy?

- Durante 15 años, Linux usó **CFS** (**Completely Fair Scheduler**): la idea era repartir la CPU de forma proporcional, dándole el procesador al proceso que **menos** tiempo virtual acumuló.
- Desde **Linux 6.6 (2023)**, CFS fue **reemplazado** por **EEVDF**: **Earliest Eligible Virtual Deadline First**.
- A cada tarea se le calcula un **virtual deadline** y se elige la tarea **elegible** con el **deadline más temprano**.

¿Y qué usa Linux hoy?

- Durante 15 años, Linux usó **CFS** (**Completely Fair Scheduler**): la idea era repartir la CPU de forma proporcional, dándole el procesador al proceso que **menos** tiempo virtual acumuló.
- Desde **Linux 6.6 (2023)**, CFS fue **reemplazado** por **EEVDF**: **Earliest Eligible Virtual Deadline First**.
- A cada tarea se le calcula un **virtual deadline** y se elige la tarea **elegible** con el **deadline más temprano**.
- Es decir: **el scheduler de propósito general de Linux hoy corre una idea con forma de EDF**, el algoritmo que acabamos de ver en la sección de tiempo real.

Scheduling en contenedores

cgroups v2

En contenedores ya no alcanza con repartir entre [procesos](#): hay que repartir entre [grupos](#) de procesos. Eso lo hacen los [cgroups](#).

Menú para hoy

- 1 Repaso
- 2 Conceptos básicos
- 3 Criterios y objetivos
- 4 Algoritmos de scheduling
- 5 Ejercicio de diseño
- 6 Scheduling de Tiempo Real
- 7 El simulador del TP
- 8 Scheduling en la vida real
- 9 Cierre**

Hoy vimos...

- Describimos varios algoritmos de scheduling de CPU.
 - First-Come, First-Served
 - Round-Robin
 - Shortest-Job-First
 - Shortest-Remaining-Time-First
 - Prioridades
 - Multilevel Queue
 - Multilevel Feedback Queue
- Comparamos los algoritmos según [throughput](#), [turnaround](#), [waiting time](#) y [response time](#).
- Aprendimos a construir [diagramas de Gantt](#) con instantes de llegada, y a leer las métricas de ahí.
- Introducimos sistemas de tiempo real
 - Soft vs Hard
 - Prioridades vs EDF (con ejemplo)
- Vimos qué se usa hoy: [EEVDF](#) en Linux y [cgroups](#) en contenedores.

Con esto se puede resolver toda la **guía práctica 2** y arrancar el **TP1**.

Lo que vimos hoy
Ráfagas de CPU e I/O
Gantt con instantes de llegada
FCFS / SJF / SRTF / RR
<i>Starvation y aging</i>
Multilevel feedback queue
EDF
Ejercicios de diseño

- Silberschatz, Galvin & Gagne. *Operating System Concepts*, Wiley. Capítulo de *CPU Scheduling*.
- Tanenbaum & Bos. *Modern Operating Systems*, Pearson.
- Arpaci-Dusseau & Arpaci-Dusseau. *Operating Systems: Three Easy Pieces* — capítulos 7, 8 y 9. **Es de donde sale el simulador del TP**, y es gratis:
<https://pages.cs.wisc.edu/~remzi/OSTEP/>

Para el que quiera ir más allá

- EEVDF en Linux: lwn.net/Articles/925371
- sched_ext (schedulers en BPF): lwn.net/Articles/922405
- CPU throttling en Kubernetes: docs.kernel.org/scheduler/sched-bwc.html